



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

KHOA CÔNG NGHỆ THÔNG TIN

HỌC PHẦN CÔNG NGHỆ PHẦN MỀM

Giảng viên: ThS. Đỗ Thị Huyền

Email: huyendt@eaut.edu.vn

4.2. CÁC HOẠT ĐỘNG CỦA QUÁ TRÌNH THIẾT KẾ PHẦN MỀM

4.4. CHẤT LƯỢNG THIẾT KẾ

4.6. THIẾT KẾ KIẾN TRÚC ỨNG DỤNG VÀ CÁC MÔ HÌNH CHO THIẾT KẾ ỨNG DỤNG



4.1. Đặc điểm của quá trình thiết kế phần mềm

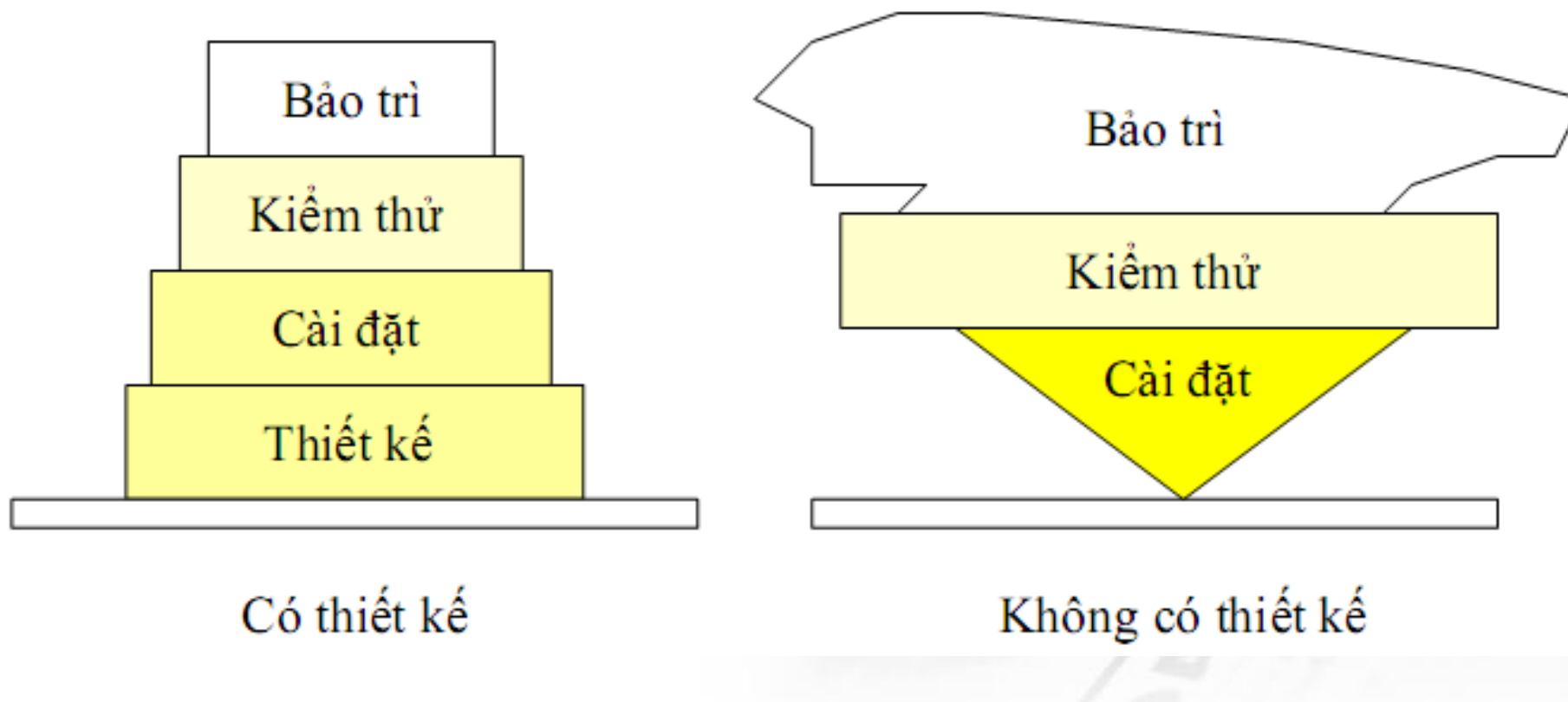


Nhiệm vụ của thiết kế là **chuyển đổi** những **yêu cầu** của hệ thống (kết quả của quá trình phân tích) sang dạng **biểu diễn của hệ thống phần mềm**. Nghĩa là xây dựng các mô tả văn bản (thiết kế chi tiết) nêu rõ mối quan hệ giữa tiên điều kiện và hậu điều kiện cho tất cả các chức năng (quá trình) của hệ thống.

4.1. Đặc điểm của quá trình thiết kế phần mềm



Tầm quan trọng thiết kế được thể hiện qua hình sau:



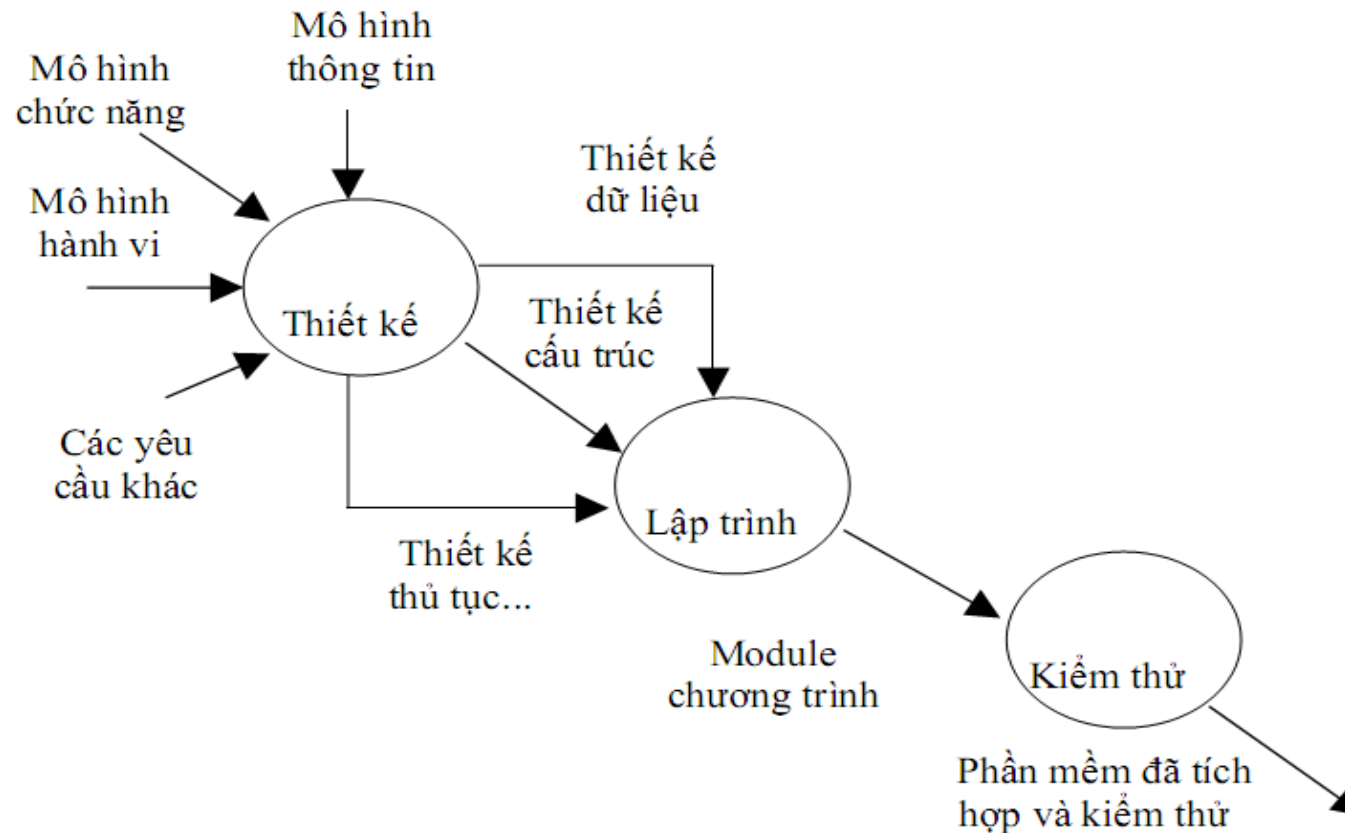
4.1. Đặc điểm của quá trình thiết kế phần mềm



- Thiết kế phải nhờ vào các kinh nghiệm và phải học tập những cái có sẵn từ các hệ thống khác; không thể chỉ đọc sách là đủ.
- Mối liên quan của thiết kế phần mềm với công nghệ phần mềm được thể hiện qua sơ đồ sau:

4.1. Đặc điểm của quá trình thiết kế phần mềm

- Luồng thông tin trong giai đoạn kỹ thuật này của tiến trình công nghệ phần mềm được minh họa trong sơ đồ sau:



4.1. Đặc điểm của quá trình thiết kế phần mềm



- Thiết kế phần mềm là hoạt động được xác lập dựa trên hai mặt: quản lý và kỹ thuật

Khía cạnh quản lý

Thiết kế sơ bộ

Thiết kế chi tiết

Khía cạnh kỹ thuật

Thiết kế dữ liệu

Thiết kế kiến trúc

Thiết kế thủ tục/đối tượng

Thiết kế giao diện

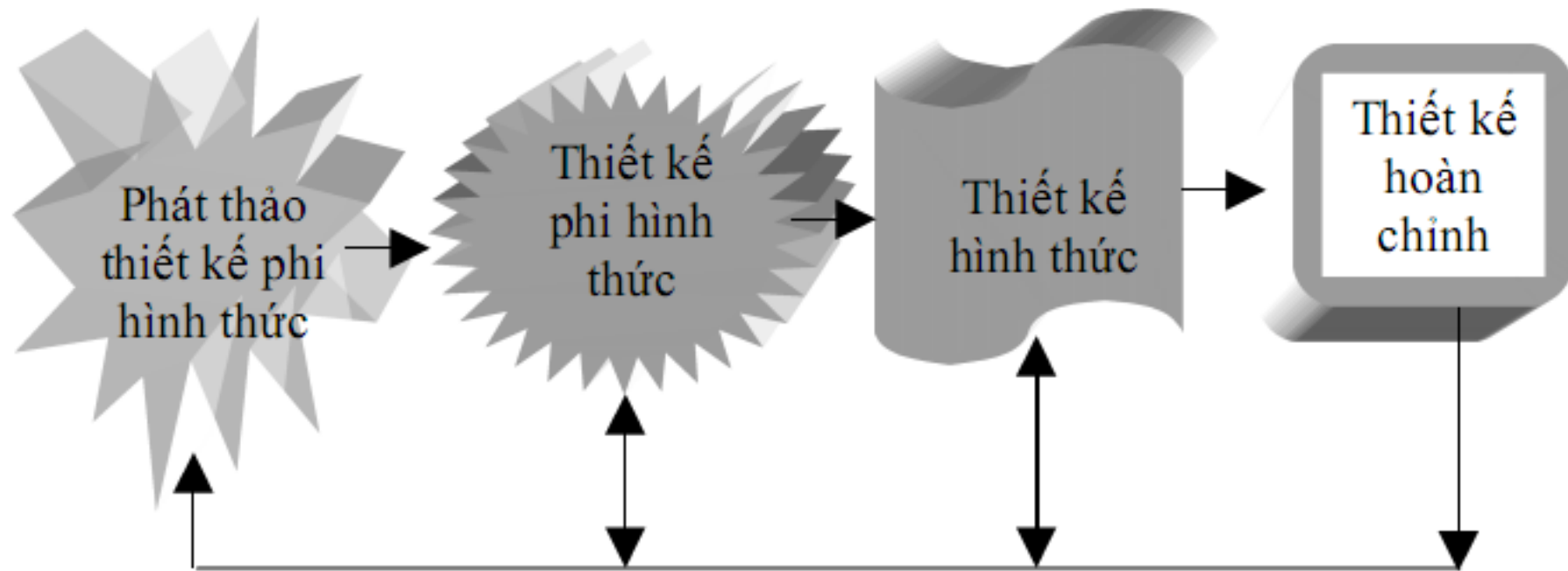
4.1. Đặc điểm của quá trình thiết kế phần mềm



- **Thiết kế sơ bộ:** quan tâm đến việc dịch các yêu cầu thành các kiến trúc dữ liệu và phần mềm.
- **Thiết kế chi tiết:** tập trung vào việc làm mịn biểu diễn kiến trúc để dẫn đến cấu trúc dữ liệu chi tiết và biểu diễn thuật toán cho phần mềm.

4.1. Đặc điểm của quá trình thiết kế phần mềm

Tiến trình thiết kế được chỉ ra ở sơ đồ sau:



4.2. Các hoạt động của quá trình thiết kế Phần mềm



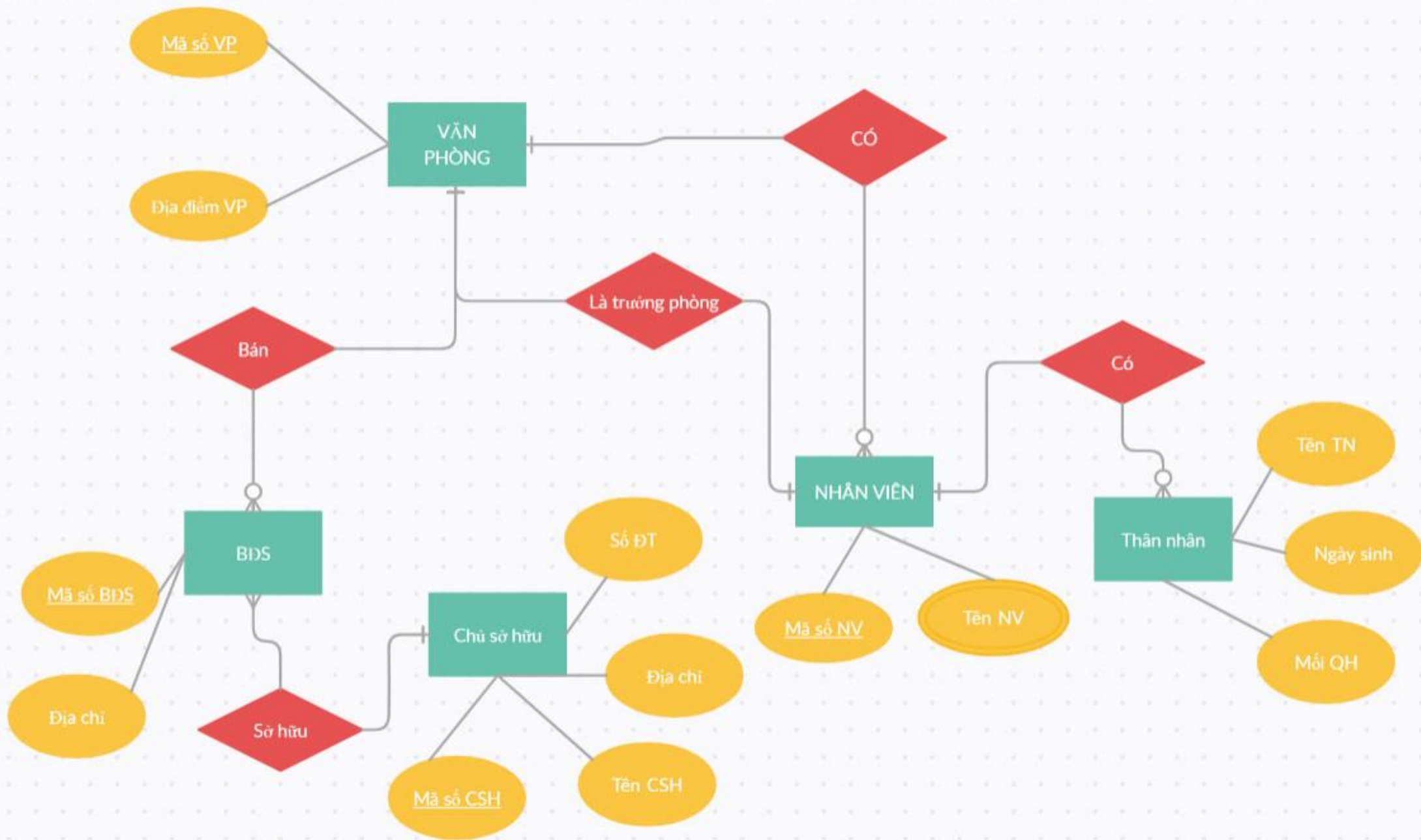
- Các hoạt động phải thực hiện trong quá trình này gồm: **định danh, suy diễn, tổng hợp lại, xem xét lại và tạo tài liệu.**
- Định danh: chính là tìm kiếm những gì quan trọng có liên quan, như việc tìm các thực thể, các đối tượng, các mối quan hệ, các chức năng, các tiến trình và các ràng buộc của hệ thống.

4.2. Các hoạt động của quá trình thiết kế Phần mềm



Một Công ty Bất Động Sản có một số văn phòng tại nhiều địa điểm. Mỗi văn phòng gồm 2 thông tin là mã số văn phòng và địa điểm đặt trụ sở.

- Mỗi văn phòng có một số nhân viên trực thuộc. Mỗi nhân viên có mã số nhân viên và tên. Với mỗi văn phòng có một nhân viên làm trưởng văn phòng. Một nhân viên có một hay nhiều thân nhân (vợ/con). Thông tin về nhân thân gồm tên, ngày sinh và mối liên hệ với nhân viên
- Công ty có danh sách các sản phẩm bất động sản cần bán. Thông tin về BĐS gồm mã số BĐS và địa chỉ. Mỗi BĐS được rao bán tại 1 và chỉ một văn phòng. Mỗi văn phòng có nhiều BĐS rao bán và có thể không có BĐS nào rao bán
- Mỗi BĐS có một chủ sở hữu. Chủ nhân được xác định bởi mã số chủ sở hữu. Một người có thể có nhiều BĐS. Thông tin về chủ sở hữu còn có tên, địa chỉ và số điện thoại



4.2. Các hoạt động của quá trình thiết kế Phần mềm



- **Suy diễn:** Xác định các chi tiết của những gì đã được định danh. Về mặt thể hiện, một yêu cầu có thể được cung cấp bản kê khai thống nhất của khách hàng cho báo cáo đặc biệt. Tìm câu trả lời cho các câu hỏi như:
 - ✓ Thông tin nào có thể đảm bảo chắc chắn về người sử dụng? Nó có đang tồn tại?
 - ✓ Điều gì đặc biệt đối với người sử dụng?
 - ✓ Kiểu của các Query người sử dụng đang hỏi?
 - ✓ Các dạng câu hỏi nào người sử dụng muốn hỏi mà họ không thể trả lời?
 - ✓ Kiểu dữ liệu phân tích nào người sử dụng cần?

4.2. Các hoạt động của quá trình thiết kế Phần mềm



➤ **Tổng hợp lại:** xây dựng một khung nhìn thống nhất của ứng dụng, điều hoà các phần không thích hợp và biểu diễn các yêu cầu trên form đồ thị. Sự biểu diễn có thể là thủ công hoặc tự động, sử dụng các công cụ tính toán cơ sở.

4.2. Các hoạt động của quá trình thiết kế Phần mềm



- **Xem xét lại:** là việc thực hiện điều khiển chất lượng. Tại cuối mỗi giai đoạn, phân tích lại tính khả thi, thời biểu và bố trí nhân sự.
- Xem xét lại chúng khi cần thiết dựa vào sự hiệu chỉnh cho hệ thống cũ, các khái niệm hiện tại của hệ thống mới.

4.2. Các hoạt động của quá trình thiết kế Phần mềm



- **Tạo tài liệu:** tạo các chương trình hữu ích đặc biệt và một tài liệu thiết kế toàn thể.
 - ✓ Tài liệu thiết kế mô tả cơ sở dữ liệu, cấu trúc của ứng dụng, các ràng buộc, cũng như các đồ thị và văn bản thiết kế.
 - ✓ Các module của chương trình bao gồm các chi tiết của tiến trình, tất cả các giao diện thiết kế và các thông tin đặc biệt để phát triển ứng dụng.

4.3. Nền tảng thiết kế



- Trừu tượng (abstraction)
- Làm mịn (Refinement)
- Tính module
- Kiến trúc phần mềm
- Che giấu thông tin
- Thiết kế module



4.3.1. Trừu tượng

- ✓ Khái niệm trừu tượng là sự cho phép tập trung vào vấn đề ở mức tổng quát nào đó, không xét tới các chi tiết mức thấp hơn không liên quan.
- ✓ Việc dùng trừu tượng hoá cho phép ta làm việc với khái niệm và thuật ngữ quen thuộc trong môi trường vấn đề mà không phải biến đổi chúng thành một cấu trúc không quen thuộc.

4.3.1. Trừu tượng

- ✓ **Trừu tượng thủ tục:** là một dãy các lệnh có tên, có một chức năng xác định và giới hạn.
- ✓ **Trừu tượng dữ liệu:** là tập hợp các dữ liệu có tên mô tả cho một sự vật dữ liệu. Đối tượng dữ liệu này thực chất là một tập hợp nhiều thông tin khác nhau và ta có thể tham khảo tới bằng cách nói tên của trừu tượng dữ liệu.
- ✓ **Trừu tượng điều khiển:** áp dụng cho cơ chế điều khiển chương trình mà không xác định các chi tiết bên trong.

4.3.2. Làm mịn

- ✓ Làm mịn là chiến lược thiết kế trên xuống. Kiến trúc của một chương trình được phát triển bằng cách các mức làm mịn liên tiếp các thủ tục. Trong mỗi bước, một hay nhiều lệnh của chương trình đã cho được phân rã thành những lệnh chi tiết hơn.
- ✓ Việc phân rã hay làm mịn liên tiếp các đặc tả này kết thúc khi tất cả các lệnh đã được diễn đạt dưới dạng bất kỳ ngôn ngữ lập trình hay ngôn ngữ máy tính nền tảng nào.
- ✓ Khi các nhiệm vụ đã được làm mịn thì dữ liệu cũng phải được làm mịn, được phân rã hay cấu trúc lại.

4.3.3. Tính Module

- ✓ Phần mềm được chia thành các phần có tên riêng biệt và định địa chỉ được, gọi là các module.
- ✓ Các module được tích hợp để thỏa mãn yêu cầu của vấn đề.

4.3.4. Che giấu thông tin

- ✓ Che giấu thông tin là khái niệm các module nên được đặc trưng bởi những quyết định thiết kế mà ẩn kín với mọi module khác, thông tin chứa trong module này không thể thâm nhập tới được từ các module khác không cần đến những thông tin đó. Che giấu thông tin kéo theo việc xác định một tập module độc lập mà trao đổi giữa các module chỉ là các thông tin thật sự cần thiết cho việc vận hành phần mềm.
- ✓ Che giấu thông tin là một tiêu chuẩn thiết kế đối với hệ thống module vì những lợi ích mà nó mang lại. Khi có sai sót xảy ra, sự thay đổi sẽ ít có khả năng lan truyền sang những vị trí khác bên trong phần mềm.

4.3.5. Thiết kế Module

- ✓Che giấu thông tin là khái niệm các module nên được đặc trưng bởi những quyết định thiết kế mà ẩn kín với mọi module khác, thông tin chứa trong module này không thể thâm nhập tới được từ các module khác không cần đến những thông tin đó. Che giấu thông tin kéo theo việc xác định một tập module độc lập mà trao đổi giữa các module chỉ là các thông tin thật sự cần thiết cho việc vận hành phần mềm.
- ✓Che giấu thông tin là một tiêu chuẩn thiết kế đối với hệ thống module vì những lợi ích mà nó mang lại. Khi có sai sót xảy ra, sự thay đổi sẽ ít có khả năng lan truyền sang những vị trí khác bên trong phần mềm.

4.4. Chất lượng thiết kế



- Sự liên kết
- Sự ghép nối
- Sự hiểu được
- Sự thích nghi được
- Một số yêu cầu thiết kế



4.4.1. Sự liên kết

- Sự liên kết của một thành phần là độ đo về tính khớp lại với nhau. Một thành phần hẳn thực hiện một chức năng logic hoặc thực hiện một thực thể logic. Tất cả các phần của thành phần đó đều tham gia vào việc thực hiện đó. Nếu một thành phần không trực tiếp tham gia vào việc chức năng logic đó thì mức độ liên kết của nó là thấp.

4.4.2. Sự ghép nối

- Ghép nối liên quan đến liên kết. Nó chỉ ra độ ghép nối giữa các đơn vị của chương trình. Hệ thống có ghép nối cao sẽ có độ ghép nối mạnh giữa các đơn vị, các đơn vị phụ thuộc lẫn nhau. Hệ thống nối ghép lỏng lẻo làm cho các đơn vị là độc lập hoặc là tương đối độc lập với nhau.
- Các module là được ghép nối chặt chẽ nếu chúng dùng các biến chung và nếu chúng trao đổi các thông tin điều khiển.

4.4.3. Sự hiểu được

Sự hiểu được liên quan tới một số các đặc trưng thành phần sau đây:

- ✓ **Tính kết dính:** Có thể hiểu được thành phần đó mà không cần tham khảo đến một thành phần nào khác hay không?
- ✓ **Đặt tên:** Phải chăng là mọi tên được dùng trong thành phần đó đều có nghĩa? Tên có nghĩa là những tên phản ánh của thực thể trong thế giới thực được mô hình bởi thành phần đó.
- ✓ **Soạn tư liệu:** Thành phần có được soạn thảo tư liệu sao cho ánh xạ giữa các thực thể của thế giới thực và thành phần đó là rõ ràng?

4.4.3. Sự hiểu được



✓ **Độ phức tạp:** độ phức tạp của các thuật toán được dùng để thực hiện thành phần đó là như thế nào? Độ phức tạp cao ám chỉ nhiều quan hệ giữa các thành phần khác nhau của thành phần thiết kế đó và một cấu trúc logic phức tạp mà nó dính líu đến độ sâu của liên kết, người thiết kế nên làm cho thiết kế thành phần càng đơn giản càng tốt.

4.4.4. Sự thích nghi được

- Cần phải dễ dàng kết hợp chặt chẽ các biến đổi về thiết kế trong toàn bộ tư liệu thiết kế. Nếu không như vậy thì các thay đổi thiết kế có thể sẽ không được đưa vào trong các mô tả liên quan. Các biến đổi tiếp sau là khó thực hiện (thành phần thiết kế này là ít thích nghi được) vì rằng sự cải biên đó không thể dựa vào tư liệu thiết kế.
- Để có độ thích nghi tối ưu thì mỗi thành phần phải là tự chứa. Một thành phần có thể là ghép nối lỏng lẻo theo nghĩa chỉ là hợp tác với các thành phần khác thông qua việc chuyển các thông báo. Điều này không giống như là tự chứa vì rằng thành phần đó có thể dựa trên các thành phần khác chẳng hạn như các chức năng hệ thống hoặc các chức năng xử lý sai. Sự thích nghi với một thành phần có thể dính líu với sự thay đổi các phần của thành phần đó mà nó dựa trên các chức năng ngoại nên đặc tả của các chức năng ngoại này cũng xét đến sự cải biên đó.
- Muốn tự chứa một cách hoàn toàn thì một thành phần không nên dùng các thành phần khác được xác định ngoại lai. Tuy nhiên, điều đó lại mâu thuẫn với kinh nghiệm nói rằng các thành phần hiện có nên là dùng lại được. Vậy là cần có một cân bằng giữa tính ưu việt của sự dùng lại các thành phần và sự mất mát tính thích nghi được của thành phần.

4.4.5. Một số yêu cầu thiết kế

- ✓ Linh hoạt đối với những yêu cầu thay đổi không định trước.
- ✓ Dễ thử nghiệm.
- ✓ Tính sáng sửa, dễ đọc
- ✓ Kích thước module nhỏ
- ✓ Tính độc lập module (tính mở/đóng giữa các module)
- ✓ Phải quan hệ chặt chẽ giữa thiết kế và yêu cầu.
- ✓ Mỗi module hoàn toàn độc lập, nó thực hiện một chức năng duy nhất và thực hiện trọn vẹn chức năng đó...

4.4.5. Một số yêu cầu thiết kế

- ✓ Thành phần trong module ràng buộc với nhau qua việc xử lý nối tiếp nhau trên cùng một dòng dữ liệu.
- ✓ Thành phần trong module được điều khiển bởi cùng một dữ liệu vào, hay cùng một phức hợp thiết bị, hay cùng thực hiện từng phần của cùng một kết xuất.
- ✓ Module có thể hiểu được hoàn toàn dựa vào những tham biến truyền cho nó và nhận từ nó.
- ✓ Khi thiết kế cố gắng giảm thiểu cấu trúc bằng việc tản ra nhiều và cố gắng co cụm khi chiều sâu tăng thêm

4.4.5. Một số yêu cầu thiết kế

- ✓ Giữ phạm vi hiệu quả của một module bên trong phạm vi kiểm soát của module đó.
- ✓ Phạm vi hiệu quả của module m được định nghĩa là tất cả các module khác bị ảnh hưởng bởi một quyết định thực hiện trong module m.
- ✓ Phạm vi kiểm soát của module m là tất cả các module và mọi module thuộc cấp và thuộc cấp cuối cùng đối với module m.

4.4.5. Một số yêu cầu thiết kế

- ✓ Ước lượng giao diện module để giảm độ phức tạp, dư thừa và tăng tính nhất quán.
- ✓ Xác định các module có chức năng dự kiến được.
- ✓ Cố gắng giữ các module một đầu vào và một đầu ra

4.5. Chiến lược thiết kế



- **Thiết kế hướng chức năng**
- **Thiết kế hướng đối tượng**



4.5.1.Thiết kế hướng chức năng

- ✓Thiết kế hướng chức năng là một cách tiếp cận thiết kế phần mềm trong đó bản thiết kế được phân giải thành một bộ các đơn thể được tác động lẫn nhau, mà một đơn thể có một chức năng được xác định rõ ràng.
- ✓Các chức năng có các trạng thái cục bộ nhưng chúng chia sẻ với nhau trạng thái hệ thống, trạng thái này là tập trung và mọi chức năng đều có thể truy cập được.

4.5.1.Thiết kế hướng chức năng

- ✓ Người ta dùng các biểu đồ dòng dữ liệu - mà nó mô tả việc xử lý dữ liệu logic, các lược đồ cấu trúc - chỉ ra cấu trúc của phần mềm và các mô tả thiết kế chi tiết.
- ✓ Cách tiếp cận chức năng để thiết kế là cách dễ nhất khi mà khối lượng thông tin trạng thái hệ thống là được làm nhỏ nhất và thông tin dùng chung nhau là rõ ràng.

4.5.2.Thiết kế hướng đối tượng

- Hệ thống được nhìn nhận như một bộ các đối tượng (chứ không phải là bộ các chức năng). Hệ thống được phân tán, mỗi đối tượng có những thông tin trạng thái riêng của nó. Đối tượng là một bộ các thuộc tính xác định trạng thái của đối tượng đó và các phép toán của nó. Nó được thừa kế từ một vài lớp đối tượng lớp cao hơn, sao cho dễ định nghĩa nó chỉ cần nêu đủ các khác nhau giữa nó và các lớp cao hơn nó.

4.5.2. Thiết kế hướng đối tượng

Thiết kế hướng đối tượng có ba đặc trưng

- ✓ Vùng dữ liệu dùng chung là bị loại bỏ. Các đối tượng liên lạc với nhau bằng cách trao đổi thông báo chứ không phải bằng các biến dùng chung.
- ✓ Các đối tượng là các thực thể độc lập mà chúng sẵn sàng được thay đổi vì rằng tất cả các trạng thái và các thông tin biểu diễn là chỉ ảnh hưởng trong phạm vi chính đối tượng đó thôi.
- ✓ Các đối tượng có thể phân tán và có thể hành động tuần tự hoặc song song. Không cần có quyết định về tính song song ngay từ một giai đoạn sớm của quá trình thiết kế.

4.5.2. Thiết kế hướng đối tượng

Ưu điểm:

- ✓ Dễ bảo trì vì các đối tượng là độc lập. Thay đổi trong thực hiện một đối tượng hoặc thêm các dịch vụ sẽ không làm ảnh hưởng tới các đối tượng hệ thống khác.
- ✓ Các đối tượng là các thành phần dùng lại được thích hợp (do tính độc lập của chúng). Một thiết kế có thể dùng lại được các đối tượng đã được thiết kế trong các bản thiết kế trước đó.
- ✓ Đối với một vài lớp hệ thống, có một phản ánh rõ ràng giữa các thực thể có thực (chẳng hạn như các thành phần phần cứng) với các đối tượng điều khiển nó trong hệ thống. Điều này cải thiện được tính dễ hiểu của thiết kế.

4.5.2.Thiết kế hướng đối tượng

Nhược điểm:

- ✓ Đây là cách nhìn thiên nhiên hệ thống là cách nhìn chức năng và việc thích nghi với cách nhìn hướng đối tượng đôi khi là khó khăn.
- ✓ Phương pháp thiết kế hướng đối tượng vẫn còn đang trong giai đoạn phát triển, còn nhiều thay đổi.

4.6. Thiết kế kiến trúc ứng dụng và các mô hình thiết kế ứng dụng



- **Thiết kế kiến trúc ứng dụng**
- **Các mô hình thiết kế ứng dụng**



4.6.1. Thiết kế kiến trúc ứng dụng

- Kiến trúc của phần mềm ứng dụng được suy dẫn ra qua tiến trình phân chia đặt mối quan hệ giữa các phần tử của giải pháp phần mềm với các bộ phận của thế giới thực được xác định không tường minh trong phân tích yêu cầu

4.6.1. Thiết kế kiến trúc ứng dụng

- ✓ Thiết kế kiến trúc ứng dụng luôn được tiến hành trước khi có các đặc tả chi tiết về hệ thống.
- ✓ Việc phân rã kiến trúc hệ thống là cần thiết cho việc cấu trúc và tổ chức đặc tả và chứa các hoạt động sau: Cấu trúc hệ thống, mô hình hóa điều khiển, phân rã mô hình.

4.6.2. Các mô hình thiết kế ứng dụng

- ***Mô hình kho dữ liệu***
- ***Mô hình client – server***
- ***Mô hình điều khiển***
- ***Mô hình đối tượng***
- ***Mô hình luồng dữ liệu***

4.6.2. Các mô hình thiết kế ứng dụng



➤ *Mô hình kho dữ liệu*

- ✓ Mỗi phân hệ duy trì một cơ sở dữ liệu riêng của mình. Dữ liệu được trao đổi giữa các phân hệ bằng cách chuyển đổi các thông báo.
- ✓ Mọi dữ liệu được lưu trữ tại một cơ sở dữ liệu trung tâm có thể được truy cập bởi mọi phân hệ. Mô hình này gọi là mô hình kho dữ liệu.
- ✓ Mô hình kho dữ liệu phù hợp cho các ứng dụng khi dữ liệu được tạo bởi một phân hệ và được sử dụng bởi các phân hệ khác. Đây là cách hữu hiệu để chia sẻ một số lượng lớn dữ liệu mà không cần chuyển đổi dữ liệu thường minh từ một phân hệ này tới các phân hệ khác.

4.6.2. Các mô hình thiết kế ứng dụng



➤ Mô hình client-server

- ✓ Mỗi phân hệ duy trì một cơ sở dữ liệu riêng của mình. Dữ liệu được trao đổi giữa các phân hệ bằng cách chuyển đổi các thông báo.
- ✓ Mọi dữ liệu được lưu trữ tại một cơ sở dữ liệu trung tâm có thể được truy cập bởi mọi phân hệ. Mô hình này gọi là mô hình kho dữ liệu.
- ✓ Mô hình kho dữ liệu phù hợp cho các ứng dụng khi dữ liệu được tạo bởi một phân hệ và được sử dụng bởi các phân hệ khác. Đây là cách hữu hiệu để chia sẻ một số lượng lớn dữ liệu mà không cần chuyển đổi dữ liệu tường minh từ một phân hệ này tới các phân hệ khác.

4.6.2. Các mô hình thiết kế ứng dụng

➤ *Mô hình đối tượng*

- ✓ Mô hình mà để vận hành, hệ thống phải được điều khiển làm việc đồng bộ và đúng. Mô hình cấu trúc không có các thông tin điều khiển mà các luồng điều khiển được chỉ ra ở mô hình điều khiển. Hai cách tiếp cận chung có thể xác định là:
- ✓ Điều khiển tập trung,
- ✓ Điều khiển trên cơ sở sự kiện
- ✓ Mô hình điều khiển bổ sung cho mô hình cấu trúc. Mỗi mô hình cấu trúc đã nói trên đều có thể dùng mô hình điều khiển tập trung hoặc mô hình điều khiển trên cơ sở sự kiện.

.

Q / A